學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

實驗報告：LAB7 Exception and System Call

一、實驗名稱

LAB7 Exception and System Call

二、實驗目的

1. 了解 Exception（例外）與 System Call（系統呼叫）的基本概念。
2. 觀察 Linux 中 system call 的執行流程。
3. 學習如何在 Linux kernel 中新增一個新的 system call。
4. 透過實作與測試，驗證新增的 system call 是否成功執行。

三、實驗原理

在 Linux 中，使用者層程式不能直接存取 kernel，因此需要透過 system call 來要求核心提供服務。system call 的本質，是由使用者程式執行特殊指令，例如 svc 0x0，觸發 exception，使 CPU 從 user space 切換到 kernel space，再由 kernel 根據 system call number 查表，找到對應的處理函式執行。

Exception 可分為三種：

（1）Fault：指令尚未完成前就發生例外，例如 Division by Zero（除以零）；處理完後，通常會重新執行原本那條指令。

（2）Trap：指令執行完成後才發生例外，例如 System Call；處理完後，會繼續執行下一條指令。

（3）Abort：嚴重錯誤，通常無法恢復，例如 Bus error；通常會直接終止 process。

學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

在 ARM64 架構下，system call 流程如下：User Space → svc 0x0 → Exception Vector → el0_svc_common() → 讀取 X8 的 syscall number → sys_call_table 查表 → 執行 sys_xxx() → 回傳結果到 X0。

四、實驗步驟

Step 1：初始化環境

先啟動 Linux kernel，並準備好 kernel source code 與 cross toolchain。本實驗使用 buildroot 提供的 source code 與 toolchain。

Kernel source code 路徑：buildroot/output/build/linux-custom

Cross toolchain 路徑：buildroot/output/host/bin/

若 buildroot 的 configure file 為 raspberrypi3_defconfig，表示為 32-bit Linux；若為 raspberrypi3_64_defconfig，則為 64-bit Linux。

Step 2：觀察一個既有的 system call

本實驗使用 getpid() 作為觀察對象，先撰寫簡單測試程式，再以 cross compiler 交叉編譯，並利用 objdump 反組譯，以觀察 system call 如何觸發 exception 進入 kernel。

test.c 內容如下：

編譯指令：./buildroot/output/host/bin/arm-linux-gcc -g -static -o test.exe test.c

反組譯指令：./buildroot/output/host/bin/arm-linux-objdump -d test.exe > assembly

之後在 assembly 中尋找 svc 或 swi 指令。觀察結果顯示：在 ARM32 中，getpid 的 syscall number 為 20；在 ARM64 中，getpid 的 syscall number 為 0xac（172）。

學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

可分別至 arch/arm/tools/syscall_32.tbl 與 arch/arm64/tools/syscall_64.tbl 進行查詢。

Step 3：新增一個 system call

本實驗以 ARM64 為例，新增一個名為 mysys 的 system call。

（1）註冊 system call：修改 arch/arm64/tools/syscall_64.tbl，加入新的 syscall 項目。

（2）宣告 system call：修改 include/linux/syscalls.h，加入 asmlinkage long sys_mysys(void);

（3）實作 system call：修改 kernel/sys.c，加入 SYSCALL_DEFINE0(mysys){ printk("Hello, I am System Call!\n"); return 0; }

Step 4：重新編譯 Linux kernel

切回 buildroot 根目錄後，執行 make linux-rebuild。編譯完成後會產生新的 Image，再將此 Image 複製到 SD card 的 BOOT（FAT partition）。

Step 5：撰寫測試程式

建立 test.c，內容以 syscall(__NR_mysys) 呼叫剛剛新增的 system call，其中 __NR_mysys 可定義為 463。若 system call 執行成功，預期回傳值為 0。

Step 6：編譯並驗證

使用 buildroot/output/host/bin/aarch64-linux-gcc -static -g test.c -o test.exe 將測試程式編譯為 ARM64 執行檔。

之後將 test.exe 複製到 Raspberry Pi 的 root filesystem partition，重新開機後在 Raspberry Pi 上執行 ./test.exe。

學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

若畫面顯示 return value 0，代表 system call 已被正常呼叫；再輸入 dmesg，若 kernel log 中出現 Hello, I am System Call!，即代表新增的 system call 成功執行。

附：實驗中使用的程式碼

1. getpid() 觀察程式

```
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    getpid();
    return 0;
}
```

2. mysys 的 kernel 實作

```
SYSCALL_DEFINE0(mysys)
{
    printk("Hello, I am System Call!\n");
    return 0;
}
```

3. 測試程式 test.c

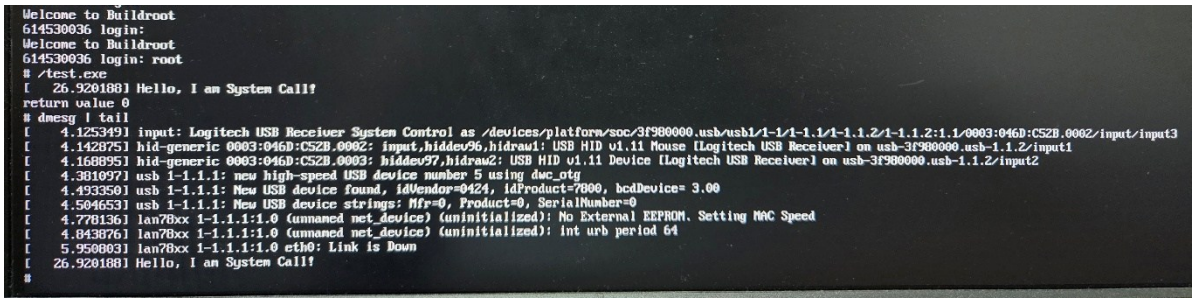
```
#include <sys/syscall.h>
#include <stdio.h>
#include <unistd.h>

#define __NR_mysys 463

int main(void)
```

學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

```
{
    long ret;
    ret = syscall(__NR_mysys);
    printf("return value %ld\n", ret);
    return 0;
}
```



五、問題與討論

Q1：請問 sys.c 中實作 mysys，SYSCALL_DEFINE0() 的作用是什麼？

SYSCALL_DEFINE0() 是 Linux kernel 用來定義 system call 的巨集。其中數字 0 代表這個 system call 沒有參數。它的作用是幫開發者建立符合 kernel 規範的 syscall handler 函式，讓這個函式能夠正確地被 system call table 呼叫。在本實驗中，SYSCALL_DEFINE0(mysys) 代表建立一個名稱為 mysys、且不接收任何參數的 system call。

Q2：asmlinkage 的描述有什麼作用？

asmlinkage 是 Linux kernel 中用來指定函式呼叫慣例的關鍵字。它的主要作用，是告訴編譯器此函式的參數應依照 kernel 規範的方式取得，確保 system call 從底層入口進入核心時，參數能被正確接收。簡單來說，asmlinkage 可以讓 system call 的函式宣告與 kernel 的低階呼叫機制一致，避免參數傳遞錯誤。